

# Microprocessors Laboratory Report

## Lab 2: Interrupt-Driven LED Control with UART Communication

---

Fraidakis Ioannis

May 2025

## 1 Introduction

---

This laboratory exercise focuses on developing an interrupt-driven LED control system that demonstrates fundamental microcontroller programming concepts. The project implements a user interface combining UART communication with physical button input to control an LED in various patterns.

The system processes numerical input where even digits trigger blinking behavior and odd digits toggle the LED state. A key feature is the ability to lock/unlock the LED via button press, preventing state changes during digit processing. The following sections detail the system architecture, implementation approach, testing methodology, and key insights gained from the development process.

## 2 System Overview

---

### 2.1 Hardware Components

The system interacts with the following hardware components:

- **LED:** Connected to GPIO pin PA\_5, used for visual feedback.
- **Button:** Connected to GPIO pin PC\_13, used for user interaction (locking/unlocking the LED). It is configured with an internal pull-up resistor and triggers an interrupt on a rising edge.

## 2.2 Software Architecture

### 2.2.1 Key Data Structures

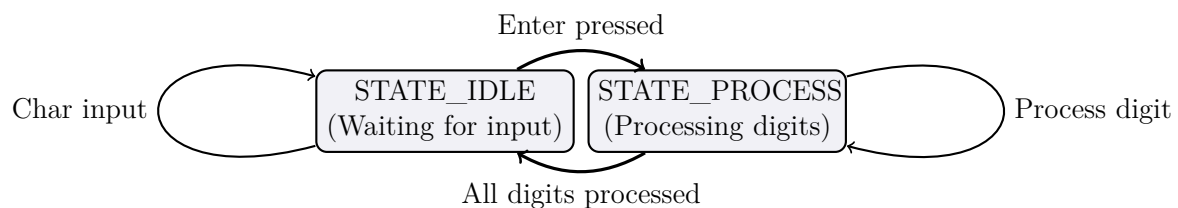
#### Data Structures

- **System (`sys_state`):** A global `volatile struct` named `sys_state` centralizes all dynamic data and operational flags. The `volatile` keyword ensures that the compiler correctly handles accesses to this structure, as it can be modified by Interrupt Service Routines (ISRs).
- **UART Receive Queue (`rx_queue`):** A queue of type `Queue` (defined in `queue.h`) is used to buffer characters received via UART. This decouples the UART ISR from the main processing loop.

### 2.2.2 State Machine

The system operates using a state machine with two main states:

- ▶ **STATE\_IDLE:** The system is waiting for user input.
- ▶ **STATE\_PROCESSING:** The system is processing a sequence of digits.



### 2.2.3 Interrupt Service Routines

Three main interrupt handlers manage system events:

- ▶ **timer\_callback():** Manages timing for LED blinking and digit processing.
- ▶ **button\_press\_callback():** Handles button press, toggling LED lock state.
- ▶ **uart\_rx\_callback():** Adds incoming UART characters to a receive queue.

### 3 Testing

| Test Scenarios  |                          |                                                 |
|-----------------|--------------------------|-------------------------------------------------|
| Action          | Input/Event              | Expected Outcome                                |
| Basic Blink     | UART: "24"               | LED blinks for digit 2, then for digit 4.       |
| Basic Toggle    | UART: "13"               | LED toggles for digit 1, then for digit 3.      |
| Mixed Sequence  | UART: "123"              | LED toggles, then blinks, then toggles.         |
| Repeat Sequence | UART: "28-"              | LED blinks for 2, then for 8, then repeats.     |
| LED Lock        | Press Button             | UART shows "LED Status: LOCKED".                |
| Empty Input     | UART: Enter              | "No input. Enter new number:" prompt.           |
| Hyphen Only     | UART: "- - -"            | "No valid digits to process." prompt.           |
| Extra Hyphens   | UART: "1-2-3" and "8-1-" | Processes as "123" and "81" (repeat).           |
| Non-Digit Chars | UART: "1a2b-c3d"         | Processes as "123".                             |
| Backspace       | UART: "123" + Backspace  | Input becomes "12", terminal cursor moves back. |
| Buffer Full     | UART: (33 digits)        | Processes first 32 digits.                      |
| Ctrl+C          | Press Ctrl+C             | Terminal clears, prompt reappears.              |
| Ctrl+X          | Press Ctrl+X             | Program termination message, LED off.           |

Table 1: Example Test Scenarios and Expected Outcomes

| Validation Steps                                                                                      |
|-------------------------------------------------------------------------------------------------------|
| ✓ Send various digit sequences and observe LED behavior and UART output.                              |
| ✓ Verify correct timing for blinking and digit processing.                                            |
| ✓ Press the button at different times (idle, during processing) and verify lock/unlock functionality. |
| ✓ Test boundary conditions like empty input, full buffer, and special characters.                     |

### 3.1 Challenges and Solutions

#### Challenges and Solutions

- 1. Keeping ISRs Short and Simple:** Interrupts can occur at any time and potentially interrupt critical operations. Long or complex code execution within ISRs can lead to system instability, missed interrupts, and timing issues. This was particularly concerning for our system with multiple interrupt sources (timer, button, UART).

*Solution:* We implemented a flag-based approach where ISRs would only update relevant status flags in the global `sys_state` structure rather than performing complex operations. For example, the button ISR simply sets `sys_state.button_event = 1` and the timer ISR sets `sys_state.digit_ready = 1` when appropriate. The main function then checks these flags during each iteration and performs the necessary processing. This keeps ISRs short and responsive while offloading complex logic to the main execution thread.

- 2. Handling New Input During Processing:** New UART inputs during digit sequence processing could interfere with ongoing operations.

*Solution:* Adopted a two-buffer strategy with reception buffer and separate processing buffer that isolates active processing from new inputs.

- 3. Button Press Detection:** We encountered a critical issue with our button interrupts. Since our button is active-low (connecting to ground when pressed), we configured it with pull-up resistors, making the pin naturally HIGH when not pressed and LOW when pressed. The problem was two-fold:
  - 1) The interrupt handler contained a condition checking `if(p->IDR&(1<<IRQ_pin_index))` that would only execute the callback when the pin was detected as HIGH
  - 2) Even when using falling edge detection (which would trigger on button press), the callback would never execute because immediately after the falling edge, the pin would be LOW, failing the HIGH-checking condition

*Solution:* We modified the GPIO driver to execute the callback regardless of pin state after an interrupt is triggered. This allowed us to properly use falling edge detection with pull-up resistors, making the system respond when the button is actually pressed (transitioning HIGH→LOW). This approach—using pull-up resistors with falling edge detection—is actually the preferred configuration for button interfaces in embedded systems (better noise immunity and responsiveness, as actions now triggered immediately on button press rather than release).